

PROJECT KNOWLEDGE BASE & TECHNICAL DOCUMENTATION

Project Title: Wireless Gesture-Controlled Smart Home & Appliance Management System

Developer / Student: GBENGA-IDOWU Afeoluwa

Matriculation Number: 21ck029291

1. PROJECT OVERVIEW & INTRODUCTION

The "Wireless Gesture-Controlled Smart Home System" is an advanced home automation project designed to allow users to control multiple high-voltage AC household appliances (Lamps, AC Fans, and Sockets) using simple, touchless hand gestures.

This project bridges the gap between low-level microcontroller electronics, wireless radio frequency (RF) communication, and high-voltage AC load management. It is split into two distinct physical modules:

1. **The Transmitter Unit (Control Interface):** A handheld or wall-mounted unit featuring an OLED display and an optical gesture sensor. It reads physical hand movements, translates them into digital commands, and wirelessly transmits these commands.
2. **The Receiver Unit (Appliance Controller):** A centralized hub connected directly to the household AC appliances via a relay module. It listens for wireless commands, toggles the respective appliances, provides audio feedback via a buzzer, and displays real-time appliance statuses on a large 20x4 LCD screen.

This documentation comprehensively covers the hardware architecture, software logic, system operations, and the specific engineering challenges (and solutions) encountered during the development phase.

2. HARDWARE ARCHITECTURE & WIRING GUIDE

To replicate, maintain, or understand this project, it is crucial to understand how the components are physically interconnected.

2.1 The Transmitter Unit (Input & Sending Hub)

Core Components:

- **Microcontroller:** Arduino (Nano/Uno).
- **Gesture Sensor:** Adafruit APDS-9960 (An advanced optical sensor that detects directional hand swipes).
- **Wireless Module:** NRF24L01 (2.4GHz Radio Frequency transceiver).
- **Display:** 1.3" OLED Display (SH1106, I2C protocol).

Wiring & Connections:

- **I2C Bus (Used by both OLED and APDS-9960):**

- **SDA (Data):** Connected to Arduino Analog Pin A4.
- **SCL (Clock):** Connected to Arduino Analog Pin A5.
- *Note:* Both the OLED and the APDS-9960 share these two pins securely because I2C uses unique hexadecimal addresses to differentiate between devices.
- **SPI Bus (Used by NRF24L01):**
 - **MOSI:** Arduino Pin 11.
 - **MISO:** Arduino Pin 12.
 - **SCK:** Arduino Pin 13.
 - **CE (Chip Enable):** Arduino Pin 9.
 - **CSN (Chip Select Not):** Arduino Pin 10.
- **Power:** The NRF24L01 is strictly powered by **3.3V** (Connecting it to 5V will instantly destroy it). The APDS-9960 and OLED are typically powered by 3.3V or 5V depending on their breakout board logic level converters.

2.2 The Receiver Unit (Output & Execution Hub)

Core Components:

- **Microcontroller:** Arduino (Nano/Uno).
- **Wireless Module:** NRF24L01 (2.4GHz Radio Frequency transceiver).
- **Display:** 2004 LCD Display (20 columns, 4 rows) with an I2C backpack.
- **Actuators:** 4-Channel 5V Relay Module (Active-LOW).
- **Feedback:** Active Buzzer.
- **Loads:** 220V-240V AC Fan (0.14A), AC Lamps, AC Socket.

Wiring & Connections:

- **Relay Control Pins:**
 - Relay 1 (Socket) -> Arduino Pin A0.
 - Relay 2 (AC Fan) -> Arduino Pin A1.
 - Relay 3 (Lamp 2) -> Arduino Pin A2.
 - Relay 4 (Lamp 1) -> Arduino Pin A3.
- **Buzzer:** Connected to Arduino Pin 2.
- **I2C Bus (LCD Display):** SDA to A4, SCL to A5.
- **SPI Bus (NRF24L01):** Same standard SPI pins as the transmitter (11, 12, 13, CE=9, CSN=10).

3. MAJOR ENGINEERING CHALLENGES & SOLUTIONS (THE "AC FAN" ISSUE)

During the development of this project, a critical, system-breaking hardware bug was discovered. The system worked perfectly when controlling lamps, but the moment the 220V AC Fan was toggled OFF, the Receiver's Arduino would freeze completely, occasionally reboot, and refuse to receive further wireless signals.

The Physics Behind the Issue: Inductive Kickback & EMI

An AC Fan is an **inductive load** because it contains a large motor consisting of coiled copper wire. When a motor is running, it stores energy in a magnetic field. When the relay suddenly turns OFF, cutting power to the fan, that magnetic field collapses instantly.

This collapse creates an enormous, high-voltage reverse electrical spike (Inductive Kickback). Inside the relay, this spike physically jumps the microscopic gap between the metal contacts, creating an invisible spark. This spark releases an **Electromagnetic Pulse (EMI)**.

This EMI blasted invisible radio noise directly into the highly sensitive NRF24L01 antenna and scrambled the logic states of the Arduino's memory, causing it to freeze or crash.

The Solution: The RC Snubber Circuit

To fix this, a hardware modification was required. We implemented an **RC Snubber**.

- **Components Used:** A 104J 630V Capacitor (0.1 μ F, rated for high voltage) and a 150-ohm Resistor.
- **Implementation:** These two components were soldered in series with each other. The combined RC Snubber was then wired in parallel across the Relay terminals (between the Common [COM] and Normally Open [NO] ports) connected to the AC Fan.
- **How it Works:** Now, when the relay clicks OFF, the massive voltage spike does not jump the gap and spark. Instead, the spike is safely "snubbed" (absorbed and dissipated as a tiny amount of heat) by the capacitor and resistor. The EMI spark is eliminated, and the microcontroller remains perfectly stable.

The NRF24L01 Auto-ACK Packet Loss Issue

Another issue encountered was packet dropping. The NRF24L01 uses an "Auto-Acknowledgment" feature where the receiver must reply with a "Got it!" signal. Because of tiny power fluctuations when relays toggle, this ACK signal was sometimes lost, making the Transmitter think the message failed.

- **The Software Fix:** In the Transmitter code, we added `radio.setRetries(15, 15);`. This command forces the Transmitter to aggressively resend the command up to 15 times with maximum delay if it does not receive an immediate acknowledgment. This ensures bulletproof communication through any minor wireless interference.

4. SOFTWARE ARCHITECTURE & FUNCTIONALITY

The codebase is structured to be non-blocking, human-readable, and highly fault-tolerant.

4.1 The Splash Screen Sequence

Both the OLED (Transmitter) and the LCD (Receiver) feature a synchronized boot-up sequence.

- Upon applying power, the displays clear and explicitly render:
 - Name: GBENGA-IDOWU
 - Afeoluwa
 - Matric: 21ck029291
- The code applies a strict delay(6000) (6 seconds) immediately after rendering this text. This ensures that any user, evaluator, or operator has ample time to read the creator's credentials without the screen rushing to the operational interface.

4.2 Transmitter Code Logic

1. **Gesture Reading:** The `apds.readGesture()` function actively scans for movement. If it detects a directional swipe, it assigns a specific action.
 - UP = Toggles Lamp 1
 - DOWN = Toggles Lamp 2
 - LEFT = Toggles Fan
 - RIGHT = Toggles Socket
2. **State Management:** The Arduino tracks the boolean state (true/false) of each device. If Lamp 1 is currently false (OFF), swiping UP changes it to true (ON), and creates the string payload "U ON".
3. **Serial Monitor Override:** The `Serial.available()` function continuously listens. If the user types a manual command like ALL ON, the code intercepts the text, uses `.toUpperCase()` to ensure case insensitivity, and processes the command just as if it were a gesture.
4. **String Packaging & Transmission:** Instead of sending single characters (which is prone to errors), the string (e.g., "U ON") is packaged into a fixed char payload[10] array. This acts as an exact 10-byte container. The `radio.write()` function securely transmits this container over the 2.4GHz frequency.
5. **OLED Updating:** The OLED dynamically updates three specific fields:
 - **Action:** Shows exactly what input was received (SWIPE UP, TYPED MANUAL, etc.).
 - **Device:** Displays the target (LAMP 1, ALL DEVICES).
 - **State:** Displays the outcome (ON or OFF).

4.3 Receiver Code Logic

1. **Continuous Listening:** The NRF24L01 is set to `radio.startListening()`. Inside the `loop()`, if `(radio.available())` triggers the moment a wireless packet is detected.
 2. **Payload Extraction:** The incoming 10-byte payload is read and converted into a standard Arduino String. The `.trim()` function is used to strip away any invisible null characters or carriage returns that might corrupt the data.
 3. **Command Parsing:** An if/else if tree analyzes the exact string. If it matches "U ON", it logically updates the internal state of Lamp 1 to true.
 4. **Hardware Execution (applyOutputs):** Because standard relay modules are **Active-LOW** (meaning sending a LOW voltage turns the relay ON), the code uses an inverted boolean structure (`digitalWrite(RELAY, !state)`). If the state is true, it sends LOW, successfully clicking the relay ON.
 5. **Feedback:** Immediately after applying the output, a 50ms stabilization delay occurs, the 2004 LCD updates its rows to display the device name and its new state, and the buzzer is driven HIGH for 100 milliseconds to emit a crisp, audible "beep."
-

5. SYSTEM OPERATIONS: HOW TO USE THE PROJECT

This section serves as the User Manual for interacting with the completed system.

What to Expect on Boot

1. Power up both the Transmitter and Receiver units.
2. **DO NOT SWIPE YET.** Both screens will display the developer's Name and Matriculation Number for exactly 6 seconds.
3. The Receiver will emit a single 150ms long beep to indicate that the RF module successfully initialized and is listening.
4. The Transmitter OLED will show Action: WAITING, SYSTEM READY, and Status: Swipe Hand!.

How to use Gestures

The APDS-9960 is an optical infrared sensor. It requires deliberate, clean movements.

1. Place your open hand approximately 2 to 4 inches directly above the gesture sensor.
2. **Swipe smoothly** across the sensor in one of the four cardinal directions (Up, Down, Left, Right).
3. **What to expect:**
 - The OLED screen on the Transmitter will instantly update to show your exact swipe direction.
 - A split second later, the Receiver will "beep".
 - You will hear a mechanical "click" from the relay.

- The AC appliance will turn on or off.
 - The large LCD screen on the Receiver will display the name of the appliance and its new state.
4. To turn that same appliance OFF, simply perform the exact same swipe gesture again. The system remembers the state and acts as a toggle.

How to use the Serial Override (Manual Control)

If you wish to control the system via a computer:

1. Connect the Transmitter Arduino to a PC via USB.
2. Open the Arduino IDE Serial Monitor (Set Baud Rate to 115200, and ensure the line ending is set to "Newline" or "Both NL & CR").
3. Type a specific command and press Enter.
 - *Valid Commands include:* U ON, U OFF, D ON, D OFF, L ON, L OFF, R ON, R OFF, ALL ON, ALL OFF.
 - *Note:* The system is case-insensitive. Typing all off will automatically be converted to ALL OFF.
4. **What to expect:** The OLED will display Action: TYPED MANUAL, and the receiver will execute the command simultaneously.

What NOT to Expect (System Limitations)

- **Near and Far Gestures:** While the Adafruit library theoretically supports proximity (Near/Far), it is highly unreliable and conflicts with the directional gesture engine. To ensure absolute reliability and no false triggers, Near and Far gestures are **not** mapped to any appliances. Attempting a "push" or "pull" motion will simply be ignored or misread as an Up/Down swipe.
- **Rapid-Fire Swiping:** If you frantically wave your hand back and forth over the sensor, the system features a built-in debounce delay (200ms) to prevent it from sending a flood of chaotic signals. You must perform distinct, separate swipes.
- **Extreme Range:** The NRF24L01 modules used (without external antennas) are designed for in-room or adjacent-room communication. Do not expect it to penetrate multiple concrete walls or function flawlessly from across a massive building without line-of-sight.

6. MAINTENANCE AND TROUBLESHOOTING

If the system behaves unexpectedly in the future, follow this diagnostic guide:

1. **Symptom:** Both screens show the boot logo, but swiping does nothing, and the OLED does not update.
 - **Cause:** The I2C wiring to the Gesture Sensor is loose.

- **Fix:** Check the SDA (A4) and SCL (A5) wires on the APDS-9960.
 - 2. **Symptom:** Swiping updates the OLED screen, but the Receiver does not beep or click the relays.
 - **Cause:** The NRF24L01 modules are not communicating.
 - **Fix:** Ensure the NRF24L01 is supplied with exactly 3.3V. Check the MOSI/MISO/SCK pins. Ensure the Receiver unit is powered on.
 - 3. **Symptom:** The Receiver receives the signal (it beeps), but the Fan or Lamp does not actually turn on.
 - **Cause:** The AC power source to the Relay COM ports is disconnected, or the appliance is unplugged.
 - 4. **Symptom:** The system works for Lamps, but completely freezes when the Fan is operated.
 - **Cause:** The RC Snubber circuit on the Fan's relay has disconnected or failed.
 - **Fix:** Inspect the 104J 630V capacitor and 150-ohm resistor connected across the Fan relay. Without this snubber, AC electromagnetic interference will crash the Arduino. Ensure it is firmly screwed into the COM and NO terminals.
-

7. CONCLUSION

The Wireless Gesture-Controlled Smart Home System successfully integrates human-computer interaction (HCI) with high-voltage automation. By migrating from single-character RF transmission to fixed 10-byte String payloads, and by addressing real-world electrical physics (Inductive kickback via RC Snubbers), the system transitioned from an unstable prototype to a robust, fault-tolerant appliance manager. The project strictly fulfills the requirements of wireless control, visual status feedback (LCD/OLED), auditory feedback (Buzzer), and personalized branding (Name and Matriculation display), resulting in a highly polished final engineering build.

CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN

Project Title: Wireless Gesture-Controlled Smart Home & Appliance Management System

Developer / Student: GBENGA-IDOWU Afeoluwa

Matriculation Number: 21ck029291

3.1 Introduction to the Methodology

This chapter details the step-by-step methodology adopted for the design, implementation, and testing of the Wireless Gesture-Controlled Smart Home System. The project was developed using a modular engineering approach, dividing the system into two distinct sub-systems: the Transmitter (Input/Control Unit) and the Receiver (Output/Execution Unit). The methodology encompasses hardware selection, circuit design, software algorithm development, and rigorous system testing to resolve real-world electrical physics challenges such as electromagnetic interference (EMI).

3.2 System Architecture and Block Design

The system architecture operates on a simplex (one-way) wireless communication topology.

1. **The Input Stage (Transmitter):** This unit acts as the primary human-machine interface (HMI). It utilizes an optical gesture sensor to capture physical hand movements, a microcontroller to process these movements into digital logic, an OLED screen for immediate visual feedback, and a 2.4GHz RF module to broadcast the commands.
2. **The Output Stage (Receiver):** This unit acts as the central hub for the home appliances. It features a matching RF module continuously listening for packets, a microcontroller to decode the payload, a 20x4 LCD and buzzer for state visualization and audio feedback, and a 4-channel relay module to switch high-voltage 220V AC loads safely.

3.3 Hardware Selection and Specification

To achieve the project objectives, specific electronic components were selected based on their reliability, compatibility, and power requirements:

- **Microcontrollers (Arduino Nano/Uno):** Selected as the central processing units for both modules due to their robust ATmega328P architecture, extensive library support, and diverse I/O capabilities (SPI, I2C, and Digital/Analog pins).
- **Gesture Sensor (Adafruit APDS-9960):** Chosen over ultrasonic sensors because it utilizes four directional photodiodes to detect specific, complex hand swipes (Up, Down, Left, Right) rather than simple proximity, allowing for multi-appliance control.
- **Wireless Transceivers (NRF24L01):** Selected for their 2.4GHz operating frequency, high baud rate capabilities (up to 2Mbps), and low power consumption. They provide reliable room-to-room wireless communication.
- **Visual Displays (1.3" OLED & 20x4 LCD):** The SH1106 OLED was chosen for the handheld transmitter due to its crisp, high-contrast visibility and low footprint. The larger 20x4 LCD with an I2C backpack was chosen for the receiver to act as a centralized dashboard, capable of displaying the user's credentials and multiple appliance states simultaneously.

- **Actuators (4-Channel 5V Relay Module):** Utilized to bridge the 5V logic of the microcontroller with the 220V-240V AC mains power required by household appliances (Socket, AC Fan, Lamps).
- **RC Snubber (104J 630V Capacitor & 150-ohm Resistor):** A critical hardware addition selected to protect the receiver circuit from inductive kickback generated by the AC Fan motor.

3.4 Hardware Implementation and Circuit Wiring

The physical assembly of the project required careful management of communication buses and power distributions to prevent component failure.

3.4.1 Transmitter Circuit Wiring

The Transmitter heavily utilizes the Inter-Integrated Circuit (I2C) and Serial Peripheral Interface (SPI) protocols to manage multiple modules without exhausting the microcontroller's pins.

- **I2C Bus Sharing:** Both the OLED display and the APDS-9960 gesture sensor require I2C communication. They were wired in parallel to the Arduino's SDA (Analog Pin 4) and SCL (Analog Pin 5). This is possible without data collision because the OLED and the APDS-9960 possess distinct hardware addresses.
- **SPI Bus:** The NRF24L01 module was wired to the standard SPI pins: MOSI (Pin 11), MISO (Pin 12), and SCK (Pin 13). The Chip Enable (CE) and Chip Select Not (CSN) were assigned to Pin 9 and Pin 10 respectively.
- **Power Distribution:** A critical design constraint was strictly powering the NRF24L01 from a clean 3.3V source, as 5V would permanently damage its internal logic. The APDS-9960 and OLED were supplied with compatible logic levels.

3.4.2 Receiver Circuit Wiring

The Receiver integrates low-voltage logic with high-voltage load switching.

- **Actuator Control:** The 4-channel relay module was connected to the Arduino's analog pins, repurposed as digital outputs: Relay 1 (Socket) to A0, Relay 2 (AC Fan) to A1, Relay 3 (Lamp 2) to A2, and Relay 4 (Lamp 1) to A3.
- **Auditory Feedback:** An active buzzer was wired to digital Pin 2 to provide physical confirmation of received signals.
- **I2C & SPI Integration:** Similar to the transmitter, the 2004 LCD was wired to the SDA (A4) and SCL (A5) pins, while the NRF24L01 utilized the SPI bus (Pins 9, 10, 11, 12, 13).
- **Appliance Integration:** The AC live wires for the Lamps, Fan, and Socket were intercepted and routed through the Common (COM) and Normally Open (NO) terminals of the respective relays, keeping the high-voltage AC physically isolated from the 5V DC Arduino logic.

3.5 Software Development and Algorithm Design

The software was developed using C++ within the Arduino Integrated Development Environment (IDE). The programming methodology focused on non-blocking execution, state tracking, and string payload formatting.

3.5.1 Initialization and Personalization (Splash Screen Sequence)

A core requirement of the software was the integration of the developer's credentials. Upon system boot, both microcontrollers execute a strict, blocking delay(6000) (6 seconds). During this window, the OLED and LCD render:

- Name: GBENGA-IDOWU
 - Afeoluwa
 - Matric: 21ck029291
- This guarantees that evaluators or users can clearly read the credentials before the screens clear and initialize the NRF24L01 radio modules and gesture sensors.

3.5.2 Transmitter Logic Flow

1. **Gesture Polling:** The `apds.readGesture()` algorithm continuously polls the optical sensor.
2. **State Toggling:** When a directional swipe (e.g., UP) is detected, the software toggles a boolean variable specific to that appliance (`stateA3 = !stateA3`).
3. **Data Packaging:** To prevent the data corruption often seen when transmitting single characters, the software translates the boolean state into a definitive string command (e.g., "U ON" or "U OFF"). This string is converted into a 10-byte character array (`char payload[10]`).
4. **Serial Override:** A secondary logic branch checks `Serial.available()`. This allows a user to type manual commands (like "ALL ON"). The software uses a `.toUpperCase()` algorithm to ensure the system is case-insensitive, seamlessly integrating manual inputs with the gesture inputs.
5. **Transmission:** The `radio.write()` function dispatches the 10-byte payload over the 2.4GHz network. The OLED screen immediately updates its UI to reflect the detected action, the target device, and the new state.

3.5.3 Receiver Logic Flow

1. **Radio Listening:** The receiver dwells in a continuous listening loop via `radio.available()`.
2. **Payload Extraction:** Upon detecting a packet, the 10-byte payload is read and immediately parsed into a standard Arduino String. The `.trim()` method is applied to strip away any invisible null characters or carriage returns that could cause a logic mismatch.
3. **Execution Tree:** An expansive if/else if algorithmic tree analyzes the string. If it matches "U ON", it updates the boolean state for Lamp 1.
4. **Active-LOW Hardware Logic:** Because the relay modules operate on Active-LOW logic, an inverted command `digitalWrite(RELAY_LAMP1, !state)` is utilized. A true state applies a LOW voltage, successfully energizing the relay coil.

5. **Feedback Loop:** After the hardware is triggered, the system applies a 50ms stabilization delay, updates the 2004 LCD rows with the appliance name and status, and pulses the buzzer HIGH for 100ms.

3.6 Testing, Troubleshooting, and the EMI Resolution

The final phase of the methodology involved rigorous empirical testing. While resistive loads (Lamps) performed flawlessly, a critical system failure was encountered when testing the inductive load (220V AC Fan, 0.14A).

The Problem: Whenever the system attempted to turn the AC Fan OFF, the receiver's microcontroller would freeze, reboot, and drop all subsequent NRF24L01 signals.

Diagnostic Process: Through isolation testing, it was determined that the issue was not software-related. Instead, it was an engineering phenomenon known as Inductive Kickback. When the relay cut power to the fan's motor, the collapsing magnetic field in the motor coil generated a massive, high-voltage reverse spike. This spike sparked across the relay contacts, emitting an Electromagnetic Pulse (EMI) that jammed the NRF24L01 radio waves and scrambled the Arduino's I/O pins.

The Methodological Solution (RC Snubber): To resolve this hardware physics issue, an RC Snubber circuit was engineered.

1. A 104J 630V (0.1 μ F) high-voltage capacitor was placed in series with a 150-ohm resistor.
 2. This circuit was wired in parallel directly across the AC Fan's relay terminals (between COM and NO).
 3. **Result:** When the relay turned OFF, the voltage spike was no longer permitted to spark. Instead, it was safely absorbed by the capacitor and dissipated as heat by the resistor. This completely eliminated the EMI, resulting in a 100% stable microcontroller and flawless wireless reception.
- Software Redundancy:** As an added layer of fault tolerance, the `radio.setRetries(15, 15)` function was added to the Transmitter. This forces the transmitter to aggressively re-transmit the payload up to 15 times if a localized EMI spark causes a momentary drop in the wireless acknowledgment packet.

CHAPTER 4: SYSTEM OPERATIONS AND USER MANUAL

(Note: This section outlines exactly how to operate the finished prototype, what to expect, and system limitations).

4.1 System Initialization

1. Apply power to both the Transmitter and Receiver units.
2. **Standby Period:** Do not attempt to swipe or send commands immediately. Both the OLED and LCD screens will exclusively display the developer's Name (GBENGA-IDOWU Afeoluwa) and Matriculation Number (21ck029291) for exactly 6 seconds.
3. Once the 6-second window elapses, the Receiver will emit a single 150ms beep, confirming the RF module has successfully initialized and is in listening mode.

4. The Transmitter OLED will transition to the control dashboard, displaying: Action: WAITING, SYSTEM READY, and Status: Swipe Hand!.

4.2 Operating via Gestures

The system utilizes optical infrared sensors, requiring deliberate and clean hand movements.

1. Hover an open hand approximately 2 to 4 inches directly above the APDS-9960 gesture sensor.
2. Swipe the hand smoothly across the sensor in one of four cardinal directions.
3. **System Responses:**
 - **Swipe UP:** Toggles Lamp 1.
 - **Swipe DOWN:** Toggles Lamp 2.
 - **Swipe LEFT:** Toggles the AC Fan.
 - **Swipe RIGHT:** Toggles the AC Socket.
4. **Feedback Sequence:** Instantly upon swiping, the OLED screen on the Transmitter will update to show the recognized direction (e.g., SWIPE UP). A fraction of a second later, the Receiver will output a short "beep," the mechanical relay will "click" to switch the 220V appliance, and the 20x4 LCD will update to display the appliance's new state (ON or OFF).
5. To turn an appliance OFF, perform the exact same swipe gesture again. The system acts as a memory toggle.

4.3 Operating via Serial Override (Computer Interface)

The system supports direct manual overriding via a computer interface.

1. Connect the Transmitter's Arduino to a PC via a USB cable.
2. Open the Arduino IDE Serial Monitor. Ensure the Baud Rate is set to 115200, and the line ending is configured to "Newline" or "Both NL & CR".
3. Type a command and press Enter. The system accepts explicitly formatted commands: U ON, U OFF, D ON, D OFF, L ON, L OFF, R ON, R OFF, ALL ON, and ALL OFF.
4. The system is programmed to auto-capitalize. Typing all on will be seamlessly converted.
5. The OLED will dynamically update its action field to read TYPED MANUAL, and the receiver will execute the command precisely as if it were a gesture.

4.4 System Limitations and Expected Constraints

To ensure proper operation, users must be aware of the following technical constraints:

- **Proximity vs. Directional Gestures:** While proximity sensing (Near and Far motions) exists in theory, it conflicts heavily with the directional gesture engine of the APDS-9960, causing high false-trigger rates. Therefore, "push" or "pull" motions are deliberately ignored by the algorithm. Only Up, Down, Left, and Right swipes are mapped to appliances.

- **Debounce Delays (Anti-Spam):** The user cannot rapidly wave their hand back and forth over the sensor. The code features a built-in 200-millisecond software debounce delay after every successful read. Users must perform distinct, separate swipes.
- **Wireless Range:** The NRF24L01 modules used in this build rely on PCB trace antennas. They are highly effective for in-room or adjacent-room communication but will experience packet loss if forced to transmit through multiple thick concrete walls or over vast outdoor distances without line-of-sight.

4.5 Maintenance Guide

If the system becomes unresponsive, operators should check the following:

1. **Blank OLED/No Gesture Detection:** Indicates a failure on the I2C bus. Check the SDA (A4) and SCL (A5) jumper wires connected to the APDS-9960 sensor.
2. **OLED Works but Receiver Does Not Beep:** Indicates an RF failure. Verify that the NRF24L01 on the Receiver is powered on, and double-check that the Transmitter's NRF24L01 is connected strictly to the 3.3V pin, not 5V.
3. **Receiver Beeps but Appliance Does Not Turn On:** Ensure that the 220V AC mains power supplying the Relay Common (COM) ports is active and securely screwed into the terminal blocks.
4. **Fan Operation Freezes System Again:** Indicates that the RC Snubber has failed, burnt out, or become disconnected. Immediately power down the 220V mains and inspect the 104J 630V capacitor and 150-ohm resistor wired across the Fan relay. Do not operate the Fan without this snubber securely in place.